

Databases Practical Assignment on File System Approach

by

(Ctr Alt Elite)

1.0 Introduction and Background

The objective of this project is to explore the limitations and complexities of early file-based data management. This exercise provides insight into the concepts of **data dependence** and **structural dependence**, two critical challenges that existed before the occurrence of modern database management systems (DBMS).

In file-based systems, **data dependence** refers to the access of data relying on the data structure, i.e., changing storage characteristics of the data will affect applications' access to the data.

Structural dependence refers to the access of a file relying on the file structure, i.e., changing file structure will affect applications' access to the file. Among other limitations of the file-based systems, the two challenges mentioned above, made early systems inflexible and time-consuming to maintain.

To explore the nature of file-based systems as used in the early information systems, we were provided with a csv (comma-separated values) formatted text file containing thousands of records about various countries and cities. The file variables include country names, city populations, landmass, independence years, languages, and life expectancy. With the provided data we aim to answer specific questions such as identifying countries ending in the letter 'a', cities with the highest population, and countries with the largest landmass. These are real-world questions that used to be computed using only basic programming logic. Now these queries are handled with ease by a DBMS using SQL.

We have chosen to use **Python** as the programming language for this assignment. This is due to its simplicity and powerful data handling capabilities. Python's built-in functions and modules offer comprehensive tools for managing data in files. These functions include `open()` for file access, string manipulation techniques and data structures for organizing records. With the help of these functions, we were able to implement the required solutions without external libraries. Requiring to use these multiple functions demonstrates the challenges of file-based data management.

Desired outputs for our program include:

- Read and interpret a structured data file.

- Filter data based on specific criteria.
- Output accurate and clear results for each of the eight sub-questions.
- Avoid repetitions in specific queries.

It is important to address this problem to develop appreciation of the historical context of file-based data systems as well as gain a practical understanding of the inefficiencies that modern database solutions have resolved. Furthermore, this project enhances our skills in file handling, condition-based filtering, string manipulation, sorting, and data aggregation using a high-level programming language.

2.0 Implementation and Testing

To answer the eight provided questions we developed a program that reads a structured text file containing country and city data. Each row of data was parsed into a dictionary. The eight questions consisted of data querying that included filtering, aggregation, sorting, and counting. The solution highlights how such tasks were handled in early file systems.

We used Python due to its readability and intuitiveness. The process we chose was plain file reading into dictionaries to simulate the manual nature of early file-based systems. We could have used pandas libraries, however, to highlight the concepts of data dependence and structural dependence, we opted not to do so.

The structure of our program includes:

- Input data file: containing thousands of rows of country and city data
- Python script:
 - Reads and parses data from input file
 - Converts each line into a dictionary of relevant fields
 - Defines separate function for each question
 - Writes answers to an output file
- Output data file: containing answers to the eight questions

Explanations of important code:

1. File Reading Pseudocode and Explanation:

Pseudocode:

Initialize an empty list with variable name “data”

Try:

Open “file.txt” in read mode with UTF-8 encoding

Skip the first line in the file

For each line in the file:

 Remove whitespace from line

 Split the line by commas into a list named “entry”

Create a dictionary called field:

 City = entry[0]

 City_population = entry[1]

 Country_name = entry[2]

 Continent = entry[3]

 Region = entry[4]

 Land_mass = entry[5]

 Indep_year = entry[6]

 Country_population = entry[7]

 Life_expectancy = entry[8]

 GNP = entry[9]

 Government_form = entry[10]

 Head_of_state = entry[11]

 Capital = entry[12]

 Language = entry[13]

 Percentage = entry[14]

Append the dictionary to the list initialised as “data”

Catch FileNotFoundError:

 Print error message

 Exit

Pseudocode Explanation:

In the first line, we begin by initializing an empty list with variable name “data”. The purpose of this is to store structured records from the file. Each entry in the dictionary represents a row/line of data from the file.

The built-in function open() allows you to open a file and return the associated file object. [1] The file is opened in read mode with UTF-8 encoding in order to guarantee the correct handling of characters. Most English text may be stored in a compact format by using UTF-8, a character encoding that uses a single 8-bit block to store common characters like those in the Latin alphabet. This ensures that the file being opened is handled properly.

We then skip the first line in the text file being opened because it contains headers rather than the actual data we are working with. Each following line in the text document is then stripped by the blank spaces and split by commas into separate values. These values are then stored in a powerful data structure that is much more efficient than lists and useful for working with large data in python, i.e. in a dictionary. It is stored in a dictionary called “field” with the corresponding headings i.e. city, city_population, country_name, continent, etc. [3] Dictionaries are highly optimized, thus performing search operations very quickly. [2] The code then appends this dictionary to the “data” list.

Key techniques involved with file handling:

- UTF-8 encoding to ensure reliable character handling
- Skipping the header for accurate data processing
- Comma-based splitting and string cleaning
- Efficient access to data by using dictionaries

2. Question Functions/Data Analysis

Each question function adheres to the same structure:

- Data transformation, sorting & filtering
- Handling of duplicates using a seen set
- Correct format of result

3. File Writing

Pseudocode:

Create a list called “questions” to format the answers in question-answer pairs

Open a new file “file2.txt” in WRITE mode as the output file

For each question-answer pair:

 Write the question label and answer to the file

 Add blank lines after each pair

End

Close output

Display confirmation message that all answers were written to “file2.txt”

Key Features:

- Question and answer pairs are stored in a list
- A new file is opened for output
- Iterates through each pair of questions and answers
- Writes the question and answer to file2.txt. A blank line is included for spacing.
- Prints message confirming that it was written to file2.txt

Code reusability:

- Helper functions (to_int, to_float) to convert values safely.
- A consistent pattern in the “top 5” queries using seen() sets to avoid duplicates.
- A single loop structure reused across multiple questions.

This modular design avoids redundancy and keeps the program organized.

Testing and solution:

Every member of the group tested the code to ensure correct and the same outputs.

Edge cases were considered such as:

- Missing values (e.g., empty population or life expectancy fields).
- Invalid numbers (e.g., non-numeric strings).
- Repeated country/city names.
- Countries with no independence year.

Code successfully:

- Answers all 8 questions
- Handles duplicate entries
- Sorts and filters data as expected
- Writes the answers clearly to an output file

Error Handling

File Handling Errors

- try-except block is included in the program to handle potential file access issues. In the event that the file is missing, an error message will be displayed before the program ends.

Data Type Conversion

- To handle conversions safely and ensure smooth data processing, the helper functions i.e. `to_int()` and `to_float()` prevent the program from crashing when being encountered by invalid or blank entries.

Data Verification

- The method `isdigit()`, used in question d and e, validates checks for the independence years before conversion. This prevents errors from occurring when non-numeric years are processed.

Programming challenges

- Ensuring the input file had consistent formatting was crucial. Any deviation (e.g. missing commas or unexpected characters) could break parsing.
- Managing data types was a common issue.
- Balancing performance while avoiding repeated computation by using the `seen()` function to avoid computing the same thing twice.

3.0 Reflections and Discussion

The programming logic relies heavily on the structure of the data file. Therefore, if any record or column/field is added to the file the indexing used in the code will be misaligned, resulting in decreased output accuracy. This limitation highlights the core issue of structural dependence.

The solution includes only the field names in the exact order that they appear in the file. If we wanted to include “GDP”, we would have to manually update the code to read and process this new field. This limitation highlights the core issue of data dependence.

Along with structural dependence and data dependence the file-based system comes with many other challenges. These include:

- Challenge:
 - Data redundancy and inconsistency: the same data may appear in multiple files and if one is updated and the other is not, the system becomes inconsistent.
- Solution:
 - With databases, redundancy is eliminated
 - Updates in one place will reflect throughout a database system
- Challenge:
 - Poor support for querying and flexibility: simple questions require manual coding
- Solution:
 - Modern databases use declarative query languages which are far more flexible. Queries can be changed without changing programming logic.

4.0 Conclusions

This project successfully highlighted the inherent limitations of early file-based management systems. By manually parsing and processing the provided data we experienced first hand the challenges that necessitated the development of modern database management systems. While the solution is effective for the eight questions, the programming logic is fragile, inflexible and simple queries require many lines of code as well as extensive error handling. Exploring this obsolete technology has provided a deeper appreciation for the sophisticated data management tools available today.

5.0 Team Members' Contribution

S/N	Name and Surname	Student Number	Key Contribution
1	Jean van Schalkwyk	4204301	Code
2	Kaamiel Isaacs	4129581	Code
3	Skye Jones (GL)	4122217	Research & Report
4	Thaakirah Mosoval	4314422	Research & Report
5	Uwais Cornelius	4121659	Code

6.0 References

- [1] Real Python, "Python's Built-in Functions," Real Python, 12 12 2023. [Online]. Available: <https://realpython.com/ref/builtin-functions/>. [Accessed 05 August 2025].
- [2] L. Cassel, Python Projects, Indianapolis, IN: Wiley, 2014.
- [3] C. P. Milliken, Python Projects for Beginners: A Ten-Week Bootcamp Approach to Python Programming, New York: Apress, 2020.

Plagiarism

We are aware and understand that plagiarism is wrong and punishable according to the UWC policy. We confirm that we have not plagiarized our work neither have we allowed anyone to copy our code or report.